Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Парфенюк Д. П.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

Задание 1:

7) Преподаватель. Класс должен обеспечивать одновременное взаимодействие с несколькими объектами класса Студент. Основные функции преподавателя – ПроверитьЛабораторнуюРаботу, ПровестиКонсультацию, ПринятьЭкзамен, ВыставитьОтметку, ПровестиЛекцию.

Решение:

```
"""
Модуль для лабораторной работы №3.
Вариант 7: Взаимодействие преподавателя и студентов.
"""
# pylint: disable=too-few-public-methods


class Student:
    """Базовый класс для представления студента."""

    def __init__(self, name):
        """Инициализация студента."""
        self.name = name

    def get_type(self):
        """Возвращает тип обучения."""
        return "Не определен"


class FullTimeStudent(Student):
    """Класс студента очного отделения."""

    def get_type(self):
        """Возвращает тип обучения."""
        return "Очник"


class PartTimeStudent(Student):
    """Класс студента заочного отделения."""

    def get_type(self):
        """Возвращает тип обучения."""
        return "Заочник"


class Teacher:
    """Класс преподавателя для взаимодействия со студентами."""

    def check_lab(self, student):
        """Проверяет лабораторную работу студента."""
        print(f"\n[!] Преподаватель проверил ЛР у {student.name}
({student.get_type()})")

    def consult(self, student):
        """Проводит консультацию для студента."""
        print(f"\n[!] Проведена консультация для {student.name}")

    def take_exam(self, student):
```

```python
        """Принимает экзамен у студента."""
        print(f"\n[!] Экзамен у {student.name} принят. Оценка выставлена.")


def main():
    """Основная функция программы с меню."""
    teacher = Teacher()
    students = []

    while True:
        print("\n--- МЕНЮ: ПРЕПОДАВАТЕЛЬ ---")
        print("1. Добавить студента-очника")
        print("2. Добавить студента-заочника")
        print("3. Проверить ЛР у всех")
        print("4. Провести консультацию")
        print("5. Принять экзамен (у последнего добавочного)")
        print("0. Выход")

        choice = input("Выберите действие: ")

        if choice == '1':
            name = input("Введите имя студента: ")
            students.append(FullTimeStudent(name))
        elif choice == '2':
            name = input("Введите имя студента: ")
            students.append(PartTimeStudent(name))
        elif choice == '3':
            for student in students:
                teacher.check_lab(student)
        elif choice == '4':
            if students:
                teacher.consult(students[-1])
        elif choice == '5':
            if students:
                teacher.take_exam(students.pop())
        elif choice == '0':
            break


if __name__ == "__main__":
    main()
```

```
--- МЕНЮ: ПРЕПОДАВАТЕЛЬ ---
1. Добавить студента-очника
2. Добавить студента-заочника
3. Проверить ЛР у всех
4. Провести консультацию
5. Принять экзамен (у последнего добавочного)
0. Выход
Выберите действие: 3

[!] Преподаватель проверил ЛР у студента Алексей (Очник)

[!] Преподаватель проверил ЛР у студента Василий (Заочник)
```

Задание 2:

7) ДУ автомобиля. Реализовать иерархию автомобилей для конкретных производителей и иерархию средств дистанционного управления. Автомобили должны иметь присущие им атрибуты и функции. ДУ имеет три основные функции – удаленная активация сигнализации, удаленное открытие/закрытие дверей и удаленный запуск двигателя. Эти функции должны отличаться по своим характеристикам для различных устройств ДУ.

Решение:

```
"""
Модуль для лабораторной работы №3.
Задание 2 (Структурный паттерн: Мост).
Вариант 7: Дистанционное управление автомобилем.
"""

# pylint: disable=too-few-public-methods


class Car:
    """Базовый класс реализации автомобиля."""

    def activate_alarm(self):
        """Метод для включения сигнализации."""

    def toggle_doors(self):
        """Метод для управления дверями."""

    def start_engine(self):
        """Метод для запуска двигателя."""


class BMW(Car):
    """Реализация функций для автомобиля марки BMW."""

    def activate_alarm(self):
        """Включение сигнализации BMW."""
        print("[BMW] Сигнализация включена: звучит громкий сигнал.")

    def toggle_doors(self):
        """Управление дверями BMW."""
        print("[BMW] Двери заблокированы/разблокированы (плавный доводчик).")

    def start_engine(self):
        """Запуск двигателя BMW."""
        print("[BMW] Двигатель запущен: V8 готов к работе.")


class Audi(Car):
    """Реализация функций для автомобиля марки Audi."""

    def activate_alarm(self):
        """Включение сигнализации Audi."""
        print("[Audi] Сигнализация активна: мигают фары.")

    def toggle_doors(self):
        """Управление дверями Audi."""
        print("[Audi] Центральный замок сработал.")

    def start_engine(self):
```

```python
        """Запуск двигателя Audi."""
        print("[Audi] Двигатель запущен кнопкой Start/Stop.")


class RemoteControl:
    """Абстракция пульта дистанционного управления."""

    def __init__(self, car):
        """Инициализация пульта конкретным автомобилем."""
        self.car = car

    def press_alarm_button(self):
        """Нажатие кнопки сигнализации."""
        self.car.activate_alarm()

    def press_door_button(self):
        """Нажатие кнопки дверей."""
        self.car.toggle_doors()

    def press_start_button(self):
        """Нажатие кнопки запуска двигателя."""
        self.car.start_engine()


def main():
    """Основная функция с интерактивным меню."""
    print("--- ВЫБОР АВТОМОБИЛЯ ---")
    print("1. Управлять BMW")
    print("2. Управлять Audi")
    brand_choice = input("Выберите марку: ")

    if brand_choice == "1":
        current_car = BMW()
    else:
        current_car = Audi()

    remote = RemoteControl(current_car)

    while True:
        print("\n--- КНОПКИ ПУЛЬТА ДУ ---")
        print("1. Сигнализация")
        print("2. Двери")
        print("3. Запуск двигателя")
        print("0. Выход")

        button = input("Нажмите кнопку на пульте: ")

        if button == "1":
            remote.press_alarm_button()
        elif button == "2":
            remote.press_door_button()
        elif button == "3":
            remote.press_start_button()
        elif button == "0":
            break
        else:
            print("Ошибка: на пульте нет такой кнопки.")


if __name__ == "__main__":
    main()
```

```
1. Управлять BMW
2. Управлять Audi
Выберите марку: 1

--- КНОПКИ ПУЛЬТА ДУ ---
1. Сигнализация
2. Двери
3. Запуск двигателя
0. Выход
Нажмите кнопку на пульте: 1
[BMW] Сигнализация включена: звучит громкий сигнал.
```

Задание 3:

7) Проект «Пиццерия». Реализовать формирование заказ(а)ов, их отмену, а также повторный заказ с теми же самыми позициями.

Решение:

```python
"""
Модуль для лабораторной работы №3.
Задание 3 (Поведенческий паттерн: Команда).
Вариант 7: Проект «Пиццерия» (Заказ, отмена, повтор).
"""

# pylint: disable=too-few-public-methods


class PizzaOrder:
    """Класс, представляющий конкретный заказ пиццы (Получатель)."""

    def __init__(self, pizza_name):
        """Инициализация заказа названием пиццы."""
        self.pizza_name = pizza_name

    def prepare(self):
        """Метод для начала приготовления пиццы."""
        print(f"-> Пицца '{self.pizza_name}' готовится в печи.")

    def cancel(self):
        """Метод для отмены приготовления."""
        print(f"-> Заказ на пиццу '{self.pizza_name}' успешно ОТМЕНЕН.")


class OrderCommand:
    """Класс команды для управления заказом."""

    def __init__(self, order):
        """Инициализация команды объектом заказа."""
        self.order = order

    def execute(self):
        """Выполнение команды (создание заказа)."""
        self.order.prepare()
```

```python
    def undo(self):
        """Отмена команды (отмена заказа)."""
        self.order.cancel()


class Waiter:
    """Класс официанта, управляющего историей заказов (Инициатор)."""

    def __init__(self):
        """Инициализация пустого списка истории."""
        self.history = []

    def place_order(self, command):
        """Принимает заказ и сохраняет его в историю."""
        command.execute()
        self.history.append(command)

    def cancel_last(self):
        """Отменяет последний сделанный заказ из истории."""
        if self.history:
            command = self.history.pop()
            command.undo()
        else:
            print("История пуста: нет заказов для отмены.")

    def repeat_last(self):
        """Повторяет последний заказ из истории."""
        if self.history:
            last_command = self.history[-1]
            print(f"ПОВТОР: {last_command.order.pizza_name}")
            self.place_order(last_command)
        else:
            print("История пуста: нет заказов для повтора.")


def main():
    """Основная функция с интерактивным меню пиццерии."""
    waiter = Waiter()

    while True:
        print("\n--- МЕНЮ ПИЦЦЕРИИ ---")
        print("1. Сделать новый заказ")
        print("2. Отменить последний заказ")
        print("3. Повторить последний заказ")
        print("0. Выход")

        choice = input("Выберите действие: ")

        if choice == "1":
            name = input("Введите название пиццы: ")
            new_order = PizzaOrder(name)
            command = OrderCommand(new_order)
            waiter.place_order(command)
        elif choice == "2":
            waiter.cancel_last()
        elif choice == "3":
            waiter.repeat_last()
        elif choice == "0":
            break
        else:
            print("Ошибка: выберите пункт из меню.")
```

```
if __name__ == "__main__":
    main()
```

```
Введите название пиццы: Мазератти 4 сыра
-> Пицца 'Мазератти 4 сыра' готовится в печи.

--- МЕНЮ ПИЦЦЕРИИ ---
1. Сделать новый заказ
2. Отменить последний заказ
3. Повторить последний заказ
0. Выход
Выберите действие: 3
ПОВТОР: Мазератти 4 сыра
-> Пицца 'Мазератти 4 сыра' готовится в печи.
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.